

FedAugment: Table Augmentation Search over Decentralized Data Repositories

Lennart Behme
BIFOLD & TU Berlin
lennart.behme@tu-berlin.de

Emil Badura
BIFOLD & TU Berlin
badura@tu-berlin.de

Leonard Geißler
BIFOLD & TU Berlin
l.geissler@tu-berlin.de

Matthias Boehm
BIFOLD & TU Berlin
matthias.boehm@tu-berlin.de

Ziawasch Abedjan
BIFOLD & TU Berlin
abedjan@tu-berlin.de

Volker Markl
BIFOLD, TU Berlin & DFKI
volker.markl@tu-berlin.de

ABSTRACT

Dataset search often aims to identify joinable or unionable datasets to augment a given query table. State-of-the-art approaches rely on large language models (LLMs) to embed tables into vector representations and perform semantic similarity search. However, existing work assumes a centralized data repository with embeddings generated by a single, homogeneous pipeline. In contrast to this simplifying assumption, data repositories in the real world are decentralized across multiple data providers, each operating their own embedding pipelines. Given the rapid pace of LLM development and provider-specific fine-tuning, enforcing a standardized pipeline is unrealistic. We introduce FedAugment, a framework for table augmentation search over decentralized data repositories with heterogeneous embeddings. FedAugment constructs a representative set of training examples, embeds it using the individual providers' pipelines, and learns projection functions that align heterogeneous embeddings into a shared vector space via multi-view contrastive learning. Using these projections, all embeddings are mapped into a globally aligned space that supports unified vector similarity search. Compared to issuing independent top-k queries to each data provider, FedAugment enables the retrieval of a global top-k result across all repositories, avoiding redundant retrievals and enabling cost-efficient table augmentation search in decentralized settings.

PVLDB Reference Format:
Lennart Behme, Emil Badura, Leonard Geißler, Matthias Boehm, Ziawasch Abedjan, and Volker Markl. FedAugment: Table Augmentation Search over Decentralized Data Repositories. PVLDB, 19(10): 2672 - 2685, 2026.
doi:10.14778/3828612.3828623

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lbhm/fedaugment.

1 INTRODUCTION

Dataset search is a foundational task and is essential for building effective data-centric machine learning (ML) pipelines, which directly depend on the relevance, coverage, and quality of the training data [6, 35, 53, 59]. A key data modality is tabular data, which plays a central role in domains such as product development, research, and decision support in finance, healthcare, and the natural sciences [10]. When working with tabular data, individual tables are frequently insufficient for comprehensive analysis or ML tasks. Instead, users must augment a given table with additional, structurally compatible tables to increase coverage, reduce sparsity, or enrich semantics. Thus, searching for augmentable, i.e., joinable or unionable, tables is a core problem in dataset search research [30, 31, 61, 68, 81].

Embedding-based Table Augmentation Search. Recent proposals use large language models (LLMs) for finding joinable or unionable tables [17, 26, 33, 37, 49]. These approaches typically fine-tune and prompt LLMs to project semantic and syntactic column features into an embedding space and define augmentation relevance via vector similarity, enabling the retrieval of relevant tables based on nearest-neighbor search. Empirical evaluations demonstrate that embedding-based techniques outperform prior work for both join and union search [24]. While effective, existing embedding-based methods make a critical simplifying assumption: all tables in a dataset corpus are embedded using a single, homogeneous embedding pipeline (including the same model, preprocessing, and prompting strategy). This assumption ensures embedding compatibility, but severely limits their application to centrally managed data repositories. In general, embedding vectors produced by different pipelines are not directly comparable, rendering cross-pipeline similarity search ill-defined. Independent data providers naturally have different preferences for their embedding pipeline, e.g., open-weight vs. proprietary or small vs. large models, making global standardization unrealistic [34, 40]. Moreover, the rapid evolution of LLMs makes it challenging to agree on a fixed embedding pipeline.

Dataset Search in Decentralized Settings. The single-pipeline assumption breaks down in decentralized data repositories, where datasets are managed by independent data providers. Decentralized data architectures are increasingly common in both academic [9, 74] and industrial contexts [21, 22]. Data providers often want to retain control over their data, driven by concerns such as privacy, intellectual property, or costs of data transfer. As a result, many decentralized datasets are not published in raw form but exposed only through metadata-based profiles to facilitate discoverability [4, 5, 57]. Acknowledging this reality, popular dataset search engines, such as Google Dataset Search [63] and Kaggle [48], rely solely on data-provider-supplied metadata to answer search queries. While effective for keyword search, these systems lack support for advanced query capabilities, especially for joinable or unionable

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 10 ISSN 2150-8097.
doi:10.14778/3828612.3828623

dataset search. The perspectives from Hulsebos et al.'s [44] survey of data professionals underscore the resulting gap. Public dataset search engines are the most popular search tool among respondents, while searching for joinable tables is also a common need. Yet current systems lack a mechanism to identify joinable or unionable tables across decentralized data repositories. We illustrate the status quo with the following example.

Example. *A sports journalist seeks to improve a soccer transfer market analysis by enriching a table with additional attributes. Lacking access to a proprietary data lake, the journalist turns to a dataset search engine that gathers dataset profiles from independent providers and searches for "soccer players". Although the top-ranked results are topically relevant, none expose tables that meaningfully overlap with the specific set of players of interest. Since the dataset profiles contain no or mutually incompatible column content synopses, the search engine cannot approximate table joinability. As a result, the system cannot effectively rank datasets by augmentation utility, and the task devolves into manual inspection and data collection.*

Limitations of Existing Work. In line with the trend toward embedding-based augmentation search [26, 33], we envision a federated setting in which local data providers expose only embeddings and pipeline metadata to a global discovery layer. This setting requires aligning multiple locally generated embeddings into a shared vector space. Prior vector space alignment techniques [45, 76] focus on pairwise alignment and do not naturally extend to aligning $n > 2$ embedding spaces produced by heterogeneous pipelines. Independently learned mappings introduce inconsistencies across pipelines and fail to produce a globally coherent representation, a problem that worsens as the number of pipelines increases.

Bridging the Gap with FedAugment. Since separately learned vector mappings are not guaranteed to be mutually consistent across more than two embedding spaces, naïvely extending existing vector alignment techniques is insufficient in our setting. To address this limitation, we introduce FedAugment, a framework for embedding-based table augmentation search over decentralized data repositories. In contrast to prior work, FedAugment eliminates the requirement for a single, globally shared embedding pipeline and instead operates on heterogeneous embeddings provided by independent data providers. A key design goal is to enable alignment under black-box constraints, assuming only inference-time access to the underlying embedding pipelines, precluding any solutions that require joint training or model introspection. At a high level, FedAugment aligns multiple heterogeneous embedding spaces into a common vector space with well-defined semantic similarity by introducing a multi-view vector alignment approach that jointly optimizes projection functions for all embedding spaces. Specifically, we construct a representative set of training samples and obtain embeddings from all pipelines via inference. We then train projection functions using a multi-view contrastive objective that maps embeddings of the same underlying input—obtained from different pipelines—onto a shared representation. This joint objective couples all embedding spaces during training, ensuring that updates to one projection are reflected across all others and thereby yielding globally consistent alignment. Finally, we index the aligned embeddings using standard approximate nearest neighbor (ANN) techniques to support efficient augmentation search.

Table 1: Notation Overview.

Symbol	Meaning	Symbol	Meaning
D	Dataset	ϕ	Augmentation scoring function
D^P	Dataset profile of D	$\mathcal{P}$	Embedding pipeline collection
$\mathcal{D}$	Data repository	$\mathcal{V}$	Embedding vector collection
$\mathcal{D}^P$	Dataset profile collection	$\mathcal{F}$	Projection function collection
C	Candidate table	c	# of candidate cols or schema size
$\mathcal{C}$	Candidate collection	d_i	Output dimension of pipeline P_i
Q	Query table	n	Number of embedding views
k	Number of search results	s	Dimension of shared vector space

Contributions. Our primary contribution is FedAugment, the first holistic framework for table augmentation search over decentralized data repositories, a key building block for scalable, next-generation dataset search. After describing basic concepts and notation in Section 2, we make the following technical contributions:

- We define the problem of table augmentation search over decentralized data repositories, introduce an embedding-based formulation of the problem, and identify key challenges as well as related design principles (Section 3).
- We present FedAugment, an end-to-end framework for joinable and unionable table search over heterogeneous embeddings, that enforces global consistency across spaces using a multi-view contrastive objective (Section 4).
- We systematically analyze data selection strategies that reduce the amount of training data needed for alignment while preserving retrieval quality (Section 4.3).
- We share the results of extensive experiments across five diverse open dataset collections, up to 12 heterogeneous embedding pipelines, and four baselines (Section 5).

2 PRELIMINARIES

We first define decentralized data repositories and table augmentation search, establishing the problem context. Table 1 summarizes key notation used throughout the paper.

Decentralized Data Repositories. A dataset D is "a collection of related observations organized [...] for a particular purpose" [13]. We consider tabular datasets, which consist of a schema $\mathcal{A} \equiv \langle A_1, \ldots, A_c \rangle$ with c columns and a set of tuples $\mathcal{T}$, where each tuple $t \in \mathcal{T}$ assigns a possibly empty value to the attributes in $\mathcal{A}$. A collection of datasets $\mathcal{D}$ forms a data repository. In a centralized setting, all available datasets reside in a single repository or data lake. In a decentralized setting, on the other hand, multiple independent data providers each maintain their own data repository. The datasets stored in these repositories are often not publicly accessible due to commercial, technical, or privacy constraints. To facilitate dataset discovery without revealing raw data, data providers publish a dataset profile D^P for each dataset D. A dataset profile comprises metadata of D, such as column headers, summary statistics, or license information [4]. These profiles allow data consumers to evaluate whether a dataset may satisfy their needs before paying for or requesting access to the underlying data. A dataset search engine aggregates dataset profiles from multiple data repositories into a collection $\mathcal{D}^P$ and answers search queries from data consumers based on the profiles. This decentralized architecture underlies major dataset search engines, such as Google Dataset Search [63], the European Union's data portal [32], and academic systems [5, 11].

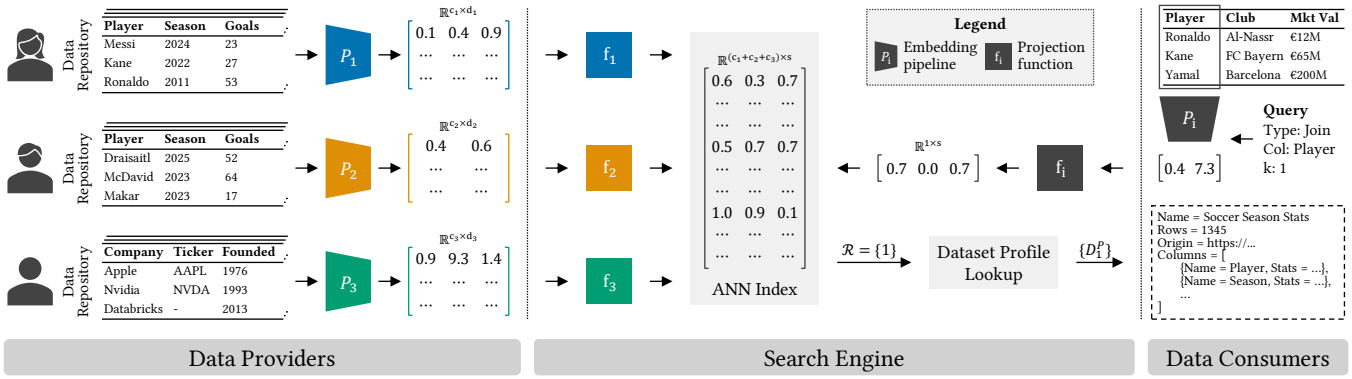


Figure 1: Overview of embedding-based table augmentation search. Data providers embed their datasets using heterogeneous pipelines. The search engine learns projection functions $\mathcal{F}$ to map embeddings to a shared vector space. A data consumer searches for datasets joinable with soccer player data and obtains results via an approximate nearest-neighbor (ANN) index lookup.

Table Augmentation Search. Complementary to basic dataset search operators, table augmentation search subsumes a class of operators that have been extensively studied in centralized settings [8, 12, 20, 26, 33, 49, 61, 68, 81]. Table augmentation comprises two main tasks, joinable and unionable table search, each with several variants (e.g., equi-, semantic-, and range-join search). Every problem variant defines an augmentation scoring function ϕ that quantifies the unionability (or joinability) between a query table Q (or column Q_i) and a candidate table C (or column C_j). For instance, a common score in equi-join search measures the cell value overlap between a column pair (Q_i, C_j). Given a query, the objective of table augmentation search is to retrieve the top-k candidate tables with the highest augmentation scores.

3 DECENTRALIZED TABLE AUGMENTATION

We now formalize the problem of decentralized table augmentation search. In contrast to centralized settings, where search algorithms operate directly on raw tables or indexes, our setting restricts access to dataset profiles only. We introduce a general formulation of table augmentation search over dataset profiles, describe how embedding-based methods instantiate this abstraction, and discuss the research challenges that arise in heterogeneous, decentralized environments.

3.1 Problem Definition

Table augmentation search aims to identify datasets that can be joined or unioned with a given query table Q. In decentralized environments, search engines do not have access to raw data but operate exclusively on dataset profiles. Each profile D^P is a nested key-value document that summarizes a dataset D through textual metadata, such as a title and description, and optional data synopses, including histograms, sketches, or column-level statistics.

Augmentation Scores. We abstract different augmentation objectives through an augmentation scoring function $\phi(Q, D^P) \to \mathbb{R}$ that approximates the utility of D for augmenting Q based on D^P. For join search, the score typically reflects the best-estimated joinability between columns of the query table Q and any column in D^P. For union search, ϕ measures how well the query table Q aligns with the table represented by D^P. Thus, our formulation unifies joinability and unionability under a common abstraction.

Problem (Table Augmentation Search Over Decentralized Data Repositories). *Given a query Q, a scoring function ϕ, and a dataset profile collection $\mathcal{D}^P$, find the top-k profiles in $\mathcal{D}^P$ with the highest augmentation scores for Q.*

Since each data provider independently generates profiles for their datasets, any profile collection gathered from multiple data repositories is highly heterogeneous [7]. Thus, the central question is how to approximately compute $\phi(Q, D^P)$ from the available information in dataset profiles while preserving augmentation accuracy.

3.2 Embedding-based Augmentation Search

Following prior work [4], we assume that data providers enrich their dataset profiles with additional synopses to improve discoverability. We instantiate the scoring function ϕ using embedding vectors. While current data repositories usually do not expose embeddings, we consider this a forward-looking yet practical setting: even when raw data are publicly available (e.g., in open data portals), precomputing and publishing embeddings enables efficient semantic retrieval. In particular, embeddings have proven effective for predicting both joinability [26] and unionability [33], while supporting efficient retrieval via ANN indexes.

System Architecture. Figure 1 continues our motivating example and shows the interaction between data providers, search engine, and data consumers in embedding-based augmentation search over decentralized data repositories. Data providers embed their datasets using embedding pipelines of their choice and publish dataset profiles containing vector representations of different dimensionality and semantics. To enable similarity search across embeddings, the search engine learns projection functions that map the individual embedding spaces into an aligned space with a well-defined similarity metric. Our sports journalist queries the search engine to find the best join candidate for the player column. The search engine projects the query embedding into the aligned space and retrieves the dataset profile of the top-1 join candidate via an ANN index lookup, identifying soccer player stats as more similar to the query than ice hockey stats, despite similar schemas.

Embeddings as a Dataset Synopsis. In the context of LLMs, an embedding vector $v \in \mathbb{R}^d$ is a fixed-size representation of an input sequence's semantic and syntactic properties. Consistent with prior

work [26, 33, 37], we assume embeddings at the column level. Nevertheless, our framework also extends to table embeddings if the desired augmentation scoring function is defined at the table level (e.g., quantifying unionability based on the cosine similarity of table embeddings). Each data provider decides (1) whether to publish embeddings at all, (2) which embedding pipeline to use, and (3) which embedding dimensionality d to employ. We make no assumptions about how embeddings are constructed, except that the generating pipeline must be inference-accessible and referenced in D^P.

Definition (Inference-Accessible Embedding Pipeline). *An embedding pipeline is a function $P : \mathcal{X} \to \mathbb{R}^d$ that transforms an input table or column $\mathcal{X}$ into a d-dimensional vector representation. The pipeline must be accessible to inference queries from the search engine, but may otherwise behave as a black box. In particular, inference access may be subject to metering, usage limits, or monetary cost.*

Heterogeneous Embeddings. Dataset profile collections may contain embeddings produced by different pipelines, requiring augmentation search to operate across heterogeneous vector spaces. Our intentionally broad definition of embedding pipelines encompasses both self-hosted pipelines based on open-weight models and commercial APIs, thereby reflecting realistic deployments. Let $\mathcal{V}$ denote all embedding vectors appearing in dataset profiles, and let $\mathcal{P}$ denote the set of embedding pipelines used across repositories. Since embeddings from different pipelines are not directly comparable, the number of views $n = |\mathcal{P}|$ quantifies the number of incompatible embedding spaces we need to align. The objective is to map embeddings from each pipeline $P_i \in \mathcal{P}$ into a shared vector space of dimension s such that semantically similar columns have high similarity, regardless of their originating pipelines. Formally, we seek a set of projection functions

$$\mathcal{F} = \{ f_i : \mathbb{R}^{d_i} \to \mathbb{R}^s \mid i = 1, \ldots, n \} \tag{1}$$

where f_i aligns the embeddings from pipeline P_i to the shared vector space. After projection, embeddings from different pipelines can be compared and ranked using a single scoring function ϕ.

Embedding-based Augmentation Scores. Our problem formulation supports any embedding-based augmentation score that is defined in terms of vector similarity. In general, embedding-based augmentation search instantiates ϕ via similarities between column-level or table-level embeddings. As a recent example of join discovery, Dong et al. [26] define ϕ as the cosine similarity between column embeddings. For union search, Fan et al. [33] measure table unionability via weighted bipartite graph matching over columns, using the pairwise cosine similarity of columns as edge weights. Both approaches achieve sublinear retrieval time with respect to the number of vectors by leveraging ANN indexes, such as HNSW [56], in combination with pruning techniques.

Augmentation Queries. At query time, data consumers can interact with the search engine in two ways. First, they may submit a raw query table, in which case the engine applies its default embedding pipeline to produce a query embedding. Second, consumers may embed their query using any pipeline $P_i \in \mathcal{P}$ for which a projection function is available; this approach maintains compatibility with the search engine while allowing consumers to leverage their own embedding infrastructure. Both options ultimately require the system to align embeddings into a shared space for comparison.

3.3 Challenges and Design Principles

Decentralized data repositories introduce challenges nonexistent in centralized table augmentation. We identify three key challenges that guide our design of a robust, scalable, and accurate solution.

Aligning Heterogeneous Embedding Pipelines. The primary challenge lies in aligning embeddings from n arbitrary embedding pipelines. Prior work on vector database integration [45, 76] focuses on pairwise alignment and does not investigate multi-view settings with $n > 2$. In contrast, our setting requires learning a family of projection functions $\mathcal{F}$ whose alignment quality and efficiency remain stable as n increases. Thus, our problem generalizes the work on embedding alignment for vector database integration. Data providers may employ fundamentally different strategies for generating embeddings, e.g., encoding entire columns as single prompts, aggregating cell-level embeddings, or using task-specific models. Moreover, embeddings are not necessarily tuned for joinable or unionable table search; providers may rely on off-the-shelf text embedding models or pipelines optimized for other tasks. This variability across embedding spaces complicates learning $\mathcal{F}$.

Minimizing Training Data Requirements. Although we assume inference-accessible embedding pipelines, generating reference embeddings from training tables incurs overhead or monetary costs. In particular, externally hosted pipelines are often metered. Here, the cost of learning projection functions increases with the volume of training data. Accordingly, the search engine should minimize the number of training columns required to learn $\mathcal{F}$.

Maintaining Search Efficiency. To ensure scalability across large numbers of data providers and highly decentralized data repositories, both the offline and online phases must be efficient. In the offline alignment phase, new embedding pipelines are expected to emerge over time, necessitating incremental updates to $\mathcal{F}$ as $|\mathcal{P}|$ increases. In the online phase, large-scale search requires sublinear retrieval time. Accordingly, the projection functions should be compatible with ANN indexes and have low overhead.

4 FEDAUGMENT

We propose FedAugment, a framework for table augmentation search over decentralized data repositories. We give an overview of FedAugment's architecture and describe its three main phases: projection model training, training data curation, and query processing.

4.1 System Overview

In decentralized data repositories with heterogeneous embedding pipelines, it is not possible to directly compute vector similarities, which is a core step in embedding-based augmentation search. FedAugment addresses this limitation by learning a set of projection functions $\mathcal{F}$, where each $f_i \in \mathcal{F}$ maps embeddings from pipeline P_i into a shared and pipeline-agnostic vector space.

Offline Stage. To learn $\mathcal{F}$, we leverage our assumption that embedding pipelines are inference-accessible as defined in Section 3.2.

- For a set of training columns, we obtain n distinct "views" per column by embedding it with all pipelines in $\mathcal{P}$. Using multi-view contrastive learning, FedAugment aligns the resulting vectors by pulling projections of the same column together while pushing projections of other columns apart.

- FedAugment reduces the time and compute cost of training by selecting representative columns from the unbounded space of possible training data. This curated subset is the basis for selecting multi-view positive and negative training samples, enabling efficient and scalable learning of $\mathcal{F}$.

Online Stage. After training, FedAugment leverages the aligned embedding space to instantiate the augmentation scoring function ϕ for join and union discovery using vector-similarity-based formulations. The aligned space enables the use of ANN indexes during query execution, ensuring sublinear retrieval time.

4.2 Projection Model Training

We begin with an intuitive overview of our training approach and subsequently detail the training data, loss formulation, aligned vector space construction, and projection model architecture.

Training Intuition. Given an n-sized set $\mathcal{P}$ of distinct embedding pipelines, our objective is to construct an s-dimensional shared vector space and learn a set of projection functions $\mathcal{F} = \{f_i\}_{i=1}^{n}$ that map each pipeline's embeddings into this aligned space. We want projections of embeddings corresponding to the *same* column, produced by different pipelines, to be close, and projections of embeddings from *different* columns to be well-separated. We measure embedding similarity using cosine similarity, although our approach generalizes to any differentiable similarity metric. To learn $\mathcal{F}$, we adopt multi-view contrastive learning [70, 73]. Contrastive learning is a branch of representation learning that guides a model to pull positive samples together in the output vector space while pushing negative samples apart [41]. Figure 2 illustrates multi-view contrastive learning and shows how each column produces n embeddings whose projections should collapse onto a common point in the aligned space. The multi-view approach provides a collective training signal across all pipelines. If the current parameters of any projection function f_i degrade alignment quality for other projections, the loss propagates this information back to f_i, enabling joint improvement of all projection functions. Prior work [70] shows that leveraging multiple views improves representation quality as the number of distinct encoders grows. The learning benefits of multiple views are especially helpful in a decentralized setting, where data providers may use various pipelines to embed their datasets.

Training Data. To train $\mathcal{F}$, search engines using FedAugment embed the columns of a reference dataset corpus using all pipelines in $\mathcal{P}$. We describe the construction of this reference corpus in Section 4.3. Let $z_{i,a} \in \mathbb{R}^{d_i}$ denote the embedding of column a produced by pipeline P_i. For a given column a, the set $\mathcal{Z}_a = \{z_{1,a}, \ldots, z_{n,a}\}$ contains its n views and forms the positive samples for contrastive learning, as all views should map to similar points in the aligned vector space. Following contrastive learning conventions, the embeddings of all other columns in the same batch serve as negative samples. The lower gray box in Figure 2 highlights the construction of $\mathcal{Z}_a$, distinguishing between positive and negative samples.

Multi-View Contrastive Loss. We now formalize our intuition into a differentiable loss function and extend the InfoNCE loss [64] to the multi-view setting. Let $\tilde{z}_{i,a} = f_i(z_{i,a})$ denote the projection of embedding $z_{i,a}$ into the aligned space. For a fixed anchor $\tilde{z}_{i,a}$, the loss aggregates similarities to all projected views of the same

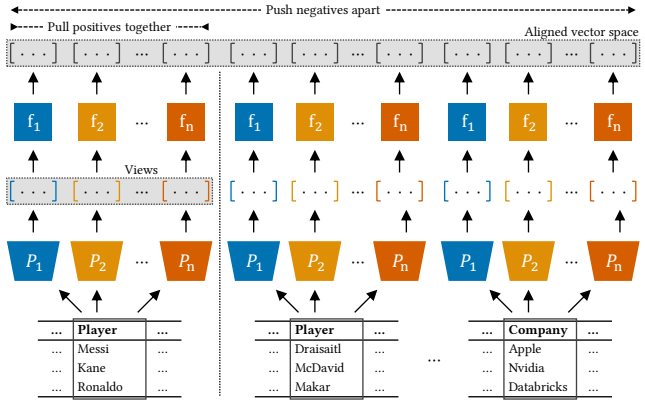


Figure 2: Multi-view contrastive learning. The lower gray box depicts how pipelines in $\mathcal{P}$ generate multiple embeddings (views) of the same underlying column. FedAugment trains $\mathcal{F}$ to map all views of a column to similar points in the aligned vector space while pushing other column projections apart.

column and normalizes over all other projections in the batch:

$$\mathcal{L}_{i,a} = -\log \frac{\sum_{j=1}^{n} \mathbb{1}_{j \neq i} \ \exp\big(\mathrm{sim}(\tilde{z}_{i,a}, \tilde{z}_{j,a})/\tau\big)}{\sum_{j=1}^{n} \sum_{b=1}^{m} \mathbb{1}_{(j,b) \neq (i,a)} \ \exp\big(\mathrm{sim}(\tilde{z}_{i,a}, \tilde{z}_{j,b})/\tau\big)} \tag{2}$$

Here, m denotes the number of distinct columns per batch, and $\tau > 0$ is a temperature controlling the concentration of the softmax distribution. The full objective averages over all anchors:

$$\mathcal{L}_{\text{InfoNCE}} = \frac{1}{nm} \sum_{i=1}^{n} \sum_{a=1}^{m} \mathcal{L}_{i,a} \tag{3}$$

Our formulation requires only weak supervision, i.e., knowledge of whether two embeddings originate from the same column. Our experiments also evaluate alternative contrastive objectives, including the normalized temperature-scaled cross-entropy [15].

Aligned Vector Space Construction. The dimensionality s of the aligned space is a key hyperparameter of FedAugment controlling both representation capacity and retrieval efficiency. Any $s > 0$ is feasible, but practical choices $s \in [\min(\mathcal{S}), \max(\mathcal{S})]$ lie within the range of input embedding sizes $\mathcal{S} = \{d_i \mid i = 1, \ldots, n\}$. Smaller values of s reduce storage costs and accelerate ANN search, whereas larger values preserve more information from high-dimensional embeddings and improve accuracy. Instead of constructing a dedicated aligned vector space, one could pick any pipeline $P_i \in \mathcal{P}$ as the reference and align other pipelines to its output space, thereby reducing the number of required projection functions by 1. However, constructing a dedicated target space allows explicit control over s and avoids privileging any data provider or embedding model.

Projection Model Architecture. Each projection function f_i is a lightweight multilayer perceptron with fully connected layers, SiLU activations [27], and batch normalization. Using independent projection heads offers two advantages over a shared encoder architecture with pipeline-specific input adapters. First, pipelines can differ substantially in their embedding geometry, normalization schemes, and representational biases. A shared encoder forces all pipelines through a single bottleneck, making it difficult to accommodate their structural heterogeneity. Second, independent heads

isolate model capacity per pipeline, thereby preventing interference during training, while the multi-view contrastive loss coordinates learning across projections to produce a coherent, shared space.

Model Maintenance. As data providers create and publish dataset profiles using new embedding pipelines, the search engine must extend $\mathcal{F}$ by training new projection heads. Our architecture naturally supports incremental updates of $\mathcal{F}$: a new projection head can be added without retraining the existing ones by freezing their weights and training only f_{n+1}. To enable cross-pipeline knowledge sharing and improve alignment consistency, the search engine can periodically unfreeze all parameters for joint training.

4.3 Training Data Curation

Before training the projection functions $\mathcal{F}$, FedAugment creates a corpus of training tables whose columns are embedded by $\mathcal{P}$ to produce the multi-view training embeddings $\mathcal{Z}$. The semantic or syntactic domain of the training columns is not critical, as long as the resulting embeddings exhibit sufficient diversity to allow the learned projection functions to generalize across heterogeneous pipelines and inputs. Hence, training data curation in FedAugment aims to maximize coverage of the input space rather than to capture domain-specific semantics (which are already captured by $\mathcal{P}$).

Candidate Collection. A straightforward approach is to collect tables from large, publicly available dataset repositories, assuming that, at sufficient scale, the resulting corpus is diverse enough. Prominent examples include the WDC Web Table Corpus [24], GitTables [43], and numerous open data portals, which together provide millions of tables and tens of millions of columns. The size of such corpora can be further increased by permuting row orders, yielding a factorial number of distinct embedding prompts from a single column. Despite this abundance of potential training data, embedding all available columns with all pipelines in $\mathcal{P}$ is infeasible in decentralized data repositories. While we assume embedding pipelines are inference-accessible, each invocation incurs a non-negligible cost, either in the form of local computational overhead for open-weight models or a direct monetary cost for API-based pipelines. Thus, we survey and empirically investigate scalable mechanisms for selecting a small but diverse subset of training columns for multi-view contrastive learning.

Curation Requirements. The most straightforward curation strategy is to randomly sample candidate columns from a uniform distribution over the candidate indices. Beyond random sampling, there is a large body of work on data-driven sample selection, known as coreset selection [36, 60]. In the context of FedAugment, suitable curation methods must satisfy three requirements: (1) they should maximize coverage of the input space to promote generalization, (2) they must scale to tens of millions of candidates, and (3) they must not require embedding the candidate columns with all pipelines in $\mathcal{P}$ or training $\mathcal{F}$ on the full candidate set, ruling out signals like gradients, hardness, or multi-view embeddings.

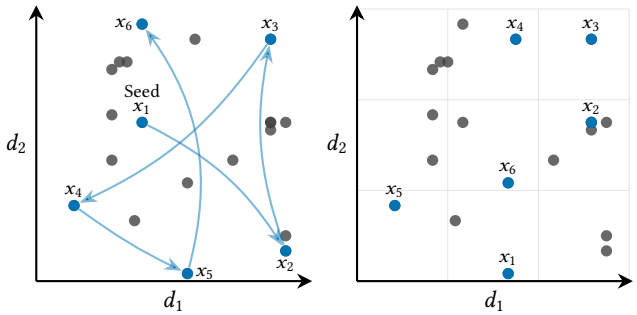
Proxy Embeddings. Although embedding all candidates with $\mathcal{P}$ is infeasible, coverage-based curation methods still require numerical representations to assess selection quality. To this end, FedAugment selects a single embedding pipeline P_p as a proxy to generate a representative embedding for each candidate column. Using proxy embeddings incurs only a one-time embedding cost per candidate, rendering curation complexity constant in the number of pipelines. Note that proxy embeddings are not part of the projection model training, but serve as an efficient surrogate to guide the curation process. By choosing an open-weight embedding model, FedAugment avoids third-party API costs during this stage. Our assumption is that semantic heterogeneity in the input is preserved across embedding spaces; while different models may induce distinct vector spaces, column dissimilarity prevails across embedding spaces. If the proxy model diverges substantially from the pipelines in $\mathcal{P}$, the curated subset may be suboptimal. However, modern LLMs exhibit high-level agreement on semantic similarity because they are pretrained on largely overlapping corpora (i.e., the internet). This claim is supported by the Platonic Representation Hypothesis [42], which states that "neural networks, trained with different objectives on different data and modalities, are converging to a shared statistical model of reality in their representation spaces".

Table 2: Complexity comparison of data curation strategies. The task is to select k out of c candidate points of dimensionality d_p. b denotes the number of buckets per dimension and t the number of k-Medoids iterations.

Algorithm	Time complexity	Space complexity	Geometry-aware
Random sampling	$O(k)$	$O(k)$	×
Grid-based sampling	$O(cd_p)$	$O(c + b^{d_p})$	✓ (weak)
Farthest-first traversal	$O(cd_pk)$	$O(c)$	✓
k-Medoids	$O(c^2d_pkt)$	$O(cd_p)$	✓

Curation Algorithms. Moser et al. [60] classify coreset selection methods into training-free and training-oriented approaches. Training-oriented methods leverage signals such as losses or gradients from model training, which requires processing the full candidate set and is therefore computationally prohibitive in our setting. FedAugment focuses on geometry-based, training-free methods that operate on proxy embeddings instead. Maximizing geometric coverage corresponds to the NP-hard metric k-center problem [75], implying that practical methods can only approximate the optimal solution. Let $\hat{Z} \in \mathbb{R}^{c \times d_p}$ denote the proxy embedding matrix generated from the candidates C, where c is the number of candidates and d_p the proxy embedding dimensionality. The goal is to select an index set $I \subseteq \{1, \ldots, c\}$ such that the row-wise submatrix $\hat{Z}_I$ maximizes coverage of the proxy embedding space. The corresponding training set $\mathcal{Z} = \mathcal{P}(C_I)$ should then enable us to learn an $\mathcal{F}$ with high-quality cross-pipeline projections. We empirically evaluate the following curation strategies, summarized in Table 2:

(1) **Random sampling** selects k candidates uniformly at random. It is computationally optimal with $O(k)$ time and space complexity, but provides no coverage guarantees.
(2) **Grid-based sampling** partitions the proxy embedding space into an equi-sized grid and samples uniformly across cells. This approach improves geometric coverage over random sampling, but without formal guarantees.
(3) **Farthest-first traversal (FFT)** greedily selects points that maximize the minimum distance to the current set of selected points, yielding a 2-approximation to the optimal k-center solution [75]. It provides strong coverage guarantees but incurs $O(cd_pk)$ time complexity. Figure 3 illustrates the differences between grid-based sampling and FFT.



(a) Farthest-first traversal **(b) Grid-based sampling**

Figure 3: Schematic comparison of data curation based on geometric coverage. (a) Farthest-first traversal iteratively selects maximally distant points. (b) Grid-based sampling uniformly draws from buckets in an equi-sized grid.

(4) ***k*-Medoids** minimizes the total distance between candidates and selected centers, favoring representativeness over geometric coverage. Its $O(c^2 d_p k t)$ time and $O(c d_p)$ space complexity render it impractical at our target scale.

Curation Parameters. The primary curation parameter is the sampling factor $\alpha = \frac{|I|}{c}$. For methods except random sampling, the proxy embedding dimensionality d_p can be reduced using techniques such as Principal Component Analysis (PCA) [46] or Matryoshka Representation Learning [51], lowering computational cost while retaining dominant variance directions.

4.4 Query Processing

At query time, data consumers submit either a raw query table or pre-computed embeddings from a pipeline $P_i \in \mathcal{P}$. If a raw table is submitted, the search engine selects an embedding pipeline-projection pair $f_i(P_i(\cdot))$ to produce a query representation in the aligned vector space. The search engine can optimize its choice through offline profiling to balance runtime (favoring lightweight pipelines) and result quality (favoring high-quality pipelines). While FedAugment could in principle extend $\mathcal{F}$ to previously unseen pipelines on demand, restricting support to a fixed set ensures predictable query latency and simplifies system design.

Query Results. Given the query embedding, FedAugment applies a vector-similarity-based augmentation scoring function to identify the k candidate tables with the highest augmentation score. We utilize recently proposed scoring functions for join [26] and union [33] discovery, which provide state-of-the-art predictions in centralized data repositories. FedAugment's architecture is orthogonal to the concrete scoring function and can incorporate future proposals.

Index-based Query Execution. A naïve execution strategy would iterate over every dataset profile $D_i^P \in \mathcal{D}^P$ to evaluate $\phi(Q, D_i^P)$. However, since the aligned embeddings $\mathcal{V}_{\mathcal{F}} \in \mathbb{R}^{|\mathcal{V}| \times s}$ lie in a unified vector space, we can use the same ANN index-based query execution strategies as prior work to achieve sublinear query execution time. As new dataset profiles arrive, the search engine incrementally inserts their embeddings into the ANN index and periodically rebuilds the data structure to sustain query performance. The choice of index structure (e.g., IVF, HNSW [56], or ScaNN [72]) is orthogonal to FedAugment, allowing system operators to trade off indexing cost, search latency, and result accuracy.

Table 3: Dataset Collection Overview.

Name	Mode	# Tables	# Columns	# Queries	AMQ†
Freyja [58]	Join	160	1,316	50	32.9
OmniMatch (City) [50]	Join	110	703	492	3.2
OmniMatch (Culture) [50]	Join	120	687	470	3.2
SANTOS (Small) [49]	Union	550	6,197	50	13.9
LakeBench (WebTable) [24]	Union‡	2,776,217	14,829,783	5,487	17.2

† AMQ: Average matches per query ‡ Deng et al. [24] also curated join labels for WebTable.

5 EXPERIMENTS

We experimentally evaluate the effectiveness and efficiency of FedAugment in three steps. After describing our setup, we share results regarding the following research questions:

- How can we minimize the amount of training data without compromising the projection quality (Section 5.2)?
- How good are FedAugment's augmentation predictions compared to existing vector database integration techniques and centralized table augmentation baselines (Section 5.3)?
- How efficient is FedAugment in terms of time and space requirements across offline and online stages (Section 5.4)?

5.1 Experiment Setup

We implemented a prototype of FedAugment using PyTorch, with an HNSW index [56] for nearest-neighbor search. All experiments were conducted on a cluster of 10 nodes running Ubuntu 22.04, equipped with two Intel Xeon Gold 6330 CPUs (112 vCores at 2.0 GHz), 512 GB RAM, and one NVIDIA A100 GPU per server.

Dataset Collections. We evaluate FedAugment on five open dataset collections to assess projection quality, augmentation prediction accuracy, and scalability. Table 3 summarizes the collections with respect to their intended augmentation mode, the number of tables and columns, the number of queries, and the average number of ground-truth matches per query. The Freyja benchmark [58] comprises 160 datasets collected from public repositories such as Kaggle and OpenML. For this corpus, the authors manually curated ground-truth labels for equi-join and semantic join relationships. The OmniMatch City and Culture collections [50] are derived from 11 and 12 source tables, respectively, obtained from the NYC Open Data Portal. To increase benchmark diversity and difficulty, the authors generated additional tables via horizontal and vertical partitioning and injected noise, including syntactic perturbations such as typos. The SANTOS benchmark [49] is based on 296 tables sourced from Canadian, UK, US, and Australian open data portals and similarly employs table partitioning to expand the benchmark set. Finally, the LakeBench WebTable benchmark [24] contains more than 2.7 million tables from the WDC Web Table Corpus and is designed for large-scale evaluation.

Embedding Pipelines. Our embedding pipeline selection aims to simulate the heterogeneity of decentralized data repositories, where pipelines are chosen independently and under varying constraints. To our knowledge, no existing join or union discovery benchmark provides heterogeneous precomputed column embeddings, so we generated them ourselves. Embedding pipelines can differ along

Table 4: Embedding Model Overview.

Model	# Parameters	Sequence length	Embedding dim.
KaLM Embedding Gemma3 [80]	11.77B	131,072	3,840
Qwen3 Embedding [79]	7.57B	40,960	4,096
Llama Embed Nemotron [2]	7.51B	131,072	4,096
Qwen3 Embedding [79]	4.02B	40,960	2,560
Qwen3 Embedding [79]	0.60B	32,768	1,024
KaLM Embedding V2.5 [80]	0.49B	32,768	896
GTR-T5-base [62]	0.11B	512	768
GTE-base [54]	0.11B	512	768
all-mpnet-base [67]	0.11B	384	768
all-distilroberta [67]	0.08B	512	768
all-MiniLM-L12 [67]	0.03B	256	384
all-MiniLM-L6 [67]	0.02B	256	384

multiple dimensions, including the embedding model, the vector dimensionality, and the prompting strategy used to serialize a column into textual input. At the same time, the landscape of embedding models is evolving rapidly. We curated a heterogeneous set of open-weight embedding models that span different generations (from recent MMTEB [29] leaders to long-standing community favorites) and hardware requirements (from 0.02 to 12 billion parameters). Table 4 summarizes the models used, reporting their parameter counts, maximum input sequence lengths, and output embedding dimensionalities. We adapted the prompting strategies from Dong et al. [26] to simulate different column-to-text serialization styles. Since the selected models span three orders of magnitude in maximum sequence length, the resulting embedding pipelines process different numbers of rows per column—another dimension of heterogeneity.

Baselines. FedAugment is the first solution for embedding-based table augmentation search over decentralized data repositories. We adapt two recently proposed methods for vector database integration to serve as baselines. LA2M [76] is based on the hypothesis that different embedding spaces are locally isometric. The method clusters training data and computes a non-parametric Procrustes alignment within each cluster to project vectors from one view into another. vec2vec [45] is an unsupervised approach based on generative adversarial networks (GANs). Its objective combines cycle-consistency constraints with an adversarial loss that encourages embeddings of the same entity, produced by different pipelines, to be indistinguishable. As additional naïve alignment baselines, we homogenize embedding dimensionalities by either truncating all embeddings to the smallest common dimension or zero-padding them to the largest dimension and then directly concatenating the resulting vectors. We refer to these approaches as Truncate and Pad. Both baselines deliberately ignore the fact that individual embedding dimensions may encode semantically incompatible information across models and thus serve as lower-bound references. To quantify the cost of decentralization, we also compare the retrieval quality of FedAugment against the centralized setting with access to raw data. We construct homogeneous embedding collections using the models in Table 4 and apply DeepJoin [26] and Starmie [33] for join and union search, respectively. We also evaluated their fine-tuning strategies, but did not observe better performance than that of the models in Table 4. To simplify reproducibility, we therefore rely on off-the-shelf embedding models from Hugging Face.

Metrics. Our primary evaluation objective is the quality of join and union predictions produced by FedAugment. We measure end-to-end effectiveness using precision@k and recall@k [24, 26, 33],

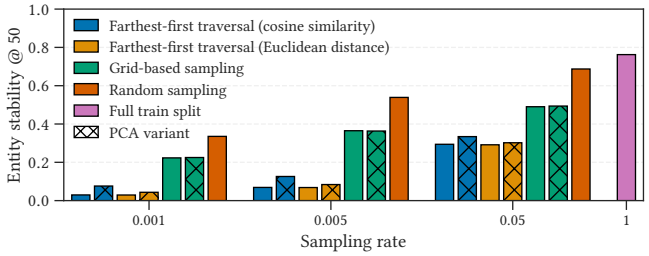


Figure 4: Comparison of data curation methods. Column subsets produced by random sampling consistently yield FedAugment models with the highest entity stability.

which quantify the accuracy and coverage of the top-k predicted augmentation candidates. In addition, we monitor the quality of projections during training by analyzing structural properties of the aligned embedding space. First, we compare the average cosine similarity across all projected embeddings with that of same-label pairs. This comparison assesses whether the contrastive learning objective successfully separates positive from negative pairs, as different views of the same column entity are expected to exhibit high mutual similarity. Second, we report entity stability@k [19], which measures the average pairwise overlap of the k-nearest-neighbor sets for all projections that originate from the same column. Entity stability assesses the consistency of the entire local neighborhood structure in the aligned space, providing a proxy for alignment quality. Finally, to assess efficiency, we measure runtime and peak memory consumption during both training and inference.

5.2 Data Curation

In the first step of our evaluation, we study data curation strategies that minimize the amount of training data required to learn projection functions. We conduct these experiments on the LakeBench WebTable corpus [24], which provides a large and diverse set of candidate columns. We partition the corpus into disjoint training, validation, and test splits containing 85%, 5%, and 10% of the tables, respectively. The training split comprises 12,609,866 columns and serves as the candidate pool for all data curation methods.

Sampling Quality. Figure 4 reports the entity stability@50 on the test split for FedAugment models trained on data subsets produced by seven curation strategies and three sampling factors, as well as for a model trained on the full training split. We could not run k-Medoids, as the method was prohibitively resource-intensive at our scale. The results show that random sampling consistently outperforms all other curation methods, refuting our initial hypothesis that strategies with geometric coverage guarantees yield better projection quality. Even at a sampling factor of 0.1%, corresponding to only 12,609 training columns, random sampling maintains a clear advantage over competing approaches. This superiority is consistent across metrics: random sampling achieves the highest entity stability at 5, 15, and 50, indicating robust vector space alignment across different neighborhood sizes. At a sampling factor of 5%, the FedAugment model trained on a random subset attains performance close to a model trained on the full training split, while reducing the data—and thus the cost—by 20×. The relatively low values of absolute entity stability stem from the highly skewed column distribution of the WebTable corpus. For example, 8,267 distinct column

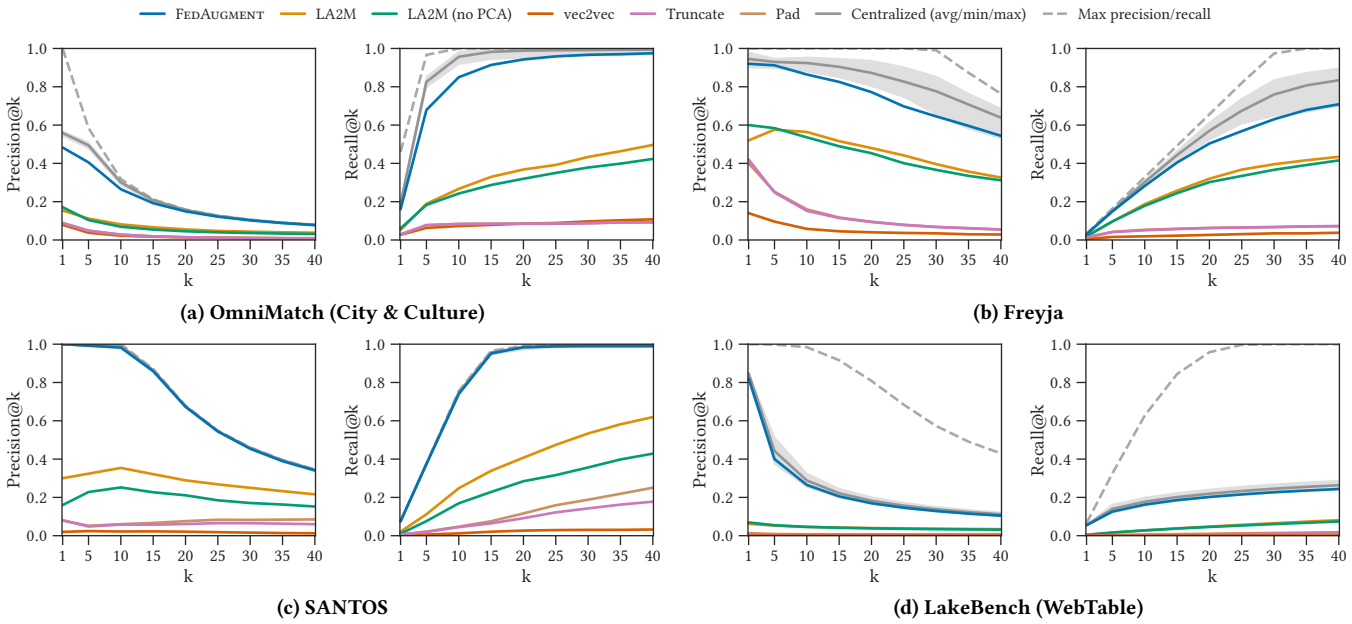


Figure 5: Recall and precision at k for FedAugment and baselines. The first row reports join predictions on OmniMatch and Freyja, each comprising candidate and query column embeddings generated by 12 distinct embedding pipelines. The second row reports union predictions on SANTOS and LakeBench WebTable, using embeddings from 12 and 8 pipelines, respectively.

names occur more than 50 times, accounting for 94.9% of all column instances. This creates large sets of semantically similar candidates, making exact neighborhood overlap difficult to achieve. For curation methods that incorporate PCA-based dimensionality reduction, we select the target dimensionality to achieve an explained variance ratio of at least 90%. However, PCA does not yield a measurable improvement in projection quality, except for FFT with cosine similarity. In addition to its strong projection quality, random sampling is by far the most efficient curation strategy. While GPU-based FFT requires approximately 10 hours at a sampling rate of 5%, random sampling reduces this process to a single index permutation followed by array slicing, completing in less than one second.

Distributional Properties. To better understand these results, we analyze the distributional properties of the curated subsets. We observe that grid sampling and FFT-based subsets exhibit an order-of-magnitude higher Wasserstein distance to the corpus than randomly sampled subsets. Moreover, we find a strong negative correlation (-0.81) between the median distance of subset points to their k nearest neighbors in the corpus and the resulting entity stability of projections. A grid search across six proxy models further shows that entity stability varies by less than 1% for grid-based sampling and by less than 3% for FFT. This observation corroborates that geometry-based methods (i) are largely insensitive to the proxy model and (ii) consistently underperform compared to random sampling. Our findings suggest that density coverage is more critical than geometric coverage for achieving high projection quality. At our scale, random sampling provides the most computationally efficient approximation of density-based sampling.

Default Configuration. Based on these insights, we used a random 5% subset of our candidate pool as the training data for all subsequent experiments. This choice ensures that model training and query evaluation are performed on disjoint dataset collections for all but one experiment. At the same time, union discovery on WebTable allows us to analyze the scenario in which training and query tables are drawn from the same distribution. A 5% sampling factor is a feasible amount of training data for realistic deployments. For example, embedding 630,493 column prompts with a maximum length of 8,192 tokens yields up to 5.165 billion tokens. As of April 2026, embedding 1 million tokens via OpenAI's batch API costs 0.01 USD, resulting in a one-time cost of 51.65 USD for an example embedding pipeline. This cost is reasonable, as we expect a dataset search engine to generate revenue from (augmentation) search requests, either directly or indirectly.

5.3 Augmentation Prediction Quality

We evaluate the augmentation prediction quality of FedAugment in comparison to our baselines using precision and recall at k on our join and union discovery benchmarks. We then analyze robustness against heterogeneous embedding characteristics along two dimensions: (i) variation in projection quality across embedding views within a fixed set of pipelines, and (ii) degradation in projection quality as the number of embedding pipelines $\mathcal{P}$ increases.

Setup. To simulate decentralized data repositories, we randomly partition each benchmark's candidate tables and queries into disjoint subsets and encode each subset using a distinct embedding pipeline. This partitioning yields 12 embedding views for all collections except LakeBench WebTable, where we restrict pipelines to fewer than 1B parameters due to WebTable's scale, resulting in 8 views. In the offline phase, we train a FedAugment model, an LA2M model, and a vec2vec model for each set of embedding views (i.e., 8 and 12 views). We then use the trained models, Pad, and Truncate to project the heterogeneous benchmark embeddings into a shared

space and build an HNSW index per model-benchmark pair. At query time, we employ each alignment model to project a query's source embedding into the aligned space, retrieve the nearest neighbors of the aligned embedding, and compute retrieval metrics.

Join Search. Figures 5a and 5b report the join prediction quality of all alignment methods on the OmniMatch and Freyja benchmark collections. For OmniMatch, we run separate experiments on the City and Culture collections but aggregate their metrics in Figure 5a for brevity. Beyond alignment methods, we report the minimum, mean, and maximum performance of DeepJoin-style predictions obtained by embedding all tables with each pipeline. This centralized baseline provides an upper bound on prediction quality. The results show that FedAugment consistently outperforms all alignment baselines for decentralized join search. Across all values of k, its precision and recall deviate by no more than 10% from the centralized DeepJoin performance and closely match the lower bound of centralized performance in most cases, indicating only a marginal quality loss despite significant heterogeneity. In contrast, all baseline alignment methods exhibit substantial quality degradation. LA2M is the strongest baseline, but still falls short. We evaluate LA2M both with and without the PCA-based dimensionality reduction and observe no meaningful differences in retrieval quality. Unlike FedAugment, which leverages a compound training signal across all embedding views, LA2M learns independent one-to-one mappings between each source view and a fixed reference view. This design can introduce redundancy and may fail to capture interactions across views, for instance, those induced by shared backbone architectures among related embedding pipelines. Moreover, LA2M does not construct a dedicated shared embedding space, limiting flexibility in selecting the dimensionality of the aligned representations. Beyond LA2M, vec2vec performs worse than the naïve Truncate and Pad baselines, suggesting that the model fails to converge due to the degree of heterogeneity and the limited training data. These findings substantiate limitations discussed by Jha et al. [45], who report convergence instabilities due to the GAN-based architecture and sensitivity to the random seed.

Union Search. Consistent with the join search results, Figures 5c and 5d show that FedAugment also outperforms all baselines for decentralized union search on the SANTOS and LakeBench WebTable collections. FedAugment not only dominates all alignment methods but often even exceeds the lowest centralized Starmie performance. On SANTOS, LA2M benefits noticeably from PCA-based dimensionality reduction, yet still trails FedAugment by a wide margin. For LakeBench WebTable, all methods, including the centralized Starmie approach, achieve lower prediction quality than on the other collections. Owing to the scale of the corpus, the WebTable authors did not annotate all valid union pairs, but applied existing union discovery techniques to generate candidate relationships, which human annotators subsequently validated. As a result, correct augmentation candidates may be missing from the ground truth, possibly leading to lower observed retrieval metrics. Nevertheless, FedAugment matches the performance of centralized union predictions, demonstrating its ability to mitigate embedding heterogeneity in large-scale, decentralized settings effectively. All in all, our experiments on join and union search show that FedAugment minimizes the quality impact of decentralization.

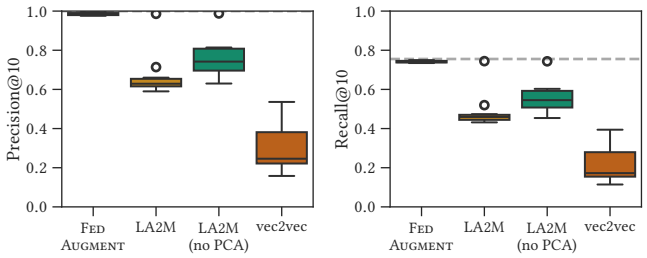


Figure 6: Projection quality per embedding view on SANTOS.

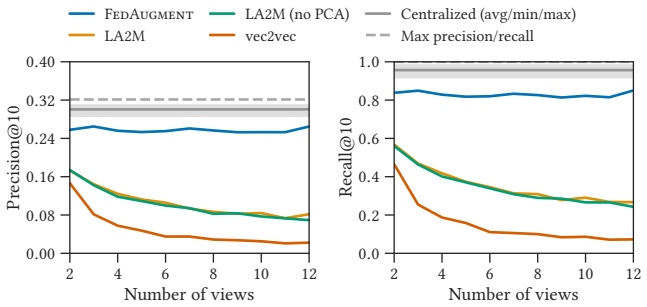


Figure 7: Join prediction quality over the number of embedding views on OmniMatch (City & Culture).

Projection Quality per View. In addition to overall prediction quality, we evaluate robustness with respect to the source embedding pipeline from which an alignment method projects. To this end, we iteratively embed all candidates and queries using a single pipeline and apply the corresponding projection function f_i to map the embeddings into the shared space. For each iteration, we compute retrieval metrics and report the distribution of precision and recall across embedding views. Figure 6 uses precision and recall@10 on SANTOS as an example to highlight that FedAugment is the most robust alignment technique, exhibiting virtually no variation in prediction quality across projection heads. In contrast, all baseline methods display substantial sensitivity to the source view, with differences of at least 30% between their best- and worst-performing views. LA2M consistently yields one near-perfect view projection because embeddings from the reference pipeline are mapped using the identity transformation. The average performance of LA2M and vec2vec is higher than in the preceding results, as this setup does not capture the scenario in which queries are projected from view i while the relevant candidates originate from different views. Both methods degrade particularly strongly when candidate and query embeddings stem from different pipelines, which is the default case in decentralized repositories. Despite this more favorable setting for the baselines, FedAugment maintains a clear advantage in prediction quality and robustness.

Projection Quality over Growing View Sets. Beyond the robustness of individual projection heads, a central source of heterogeneity in decentralized repositories is the number of distinct embedding views (i.e., the cardinality of $\mathcal{P}$). Our two main baselines, LA2M and vec2vec, are evaluated only on pairs of embedding pipelines in their original work [45, 76]. To study the scalability of these baselines and FedAugment with respect to view diversity, we vary the number of embedding pipelines from 2 to 12 by iteratively adding a new

Table 5: Efficiency comparison of projection models trained on 12 views. Inference time is measured with a batch size of 32 to reflect low-latency scenarios. Space denotes the peak GPU memory consumption during training or inference.

	Training		Inference	
Model	Time (h)	Space (GB)	Time (ms)	Space (GB)
FedAugment	3.6	49.6	0.2	0.2
LA2M	0.7	52.2	3.6	0.2
LA2M (no PCA)	4.4	65.3	3.3	17.1
vec2vec	> 24	37.2	0.8	1.0

pipeline with increasing embedding dimension to the set. For each n, we first train a projection model per alignment technique. Then, we partition the dataset collection into n uniformly sized random chunks, embed each chunk using a different pipeline, and align all embeddings into a shared vector space. Figure 7 reports the aggregated precision and recall@10 on OmniMatch City and Culture, showing the centralized performance range of all 12 pipelines as a reference. While FedAugment maintains stable prediction quality as the number of embedding views increases, all baselines exhibit a pronounced degradation with growing $|\mathcal{P}|$. When increasing the number of views from 2 to 12, the precision and recall of LA2M and vec2vec drop by more than half. These results provide further empirical evidence that existing alignment baselines do not scale to the level of embedding heterogeneity encountered in decentralized data repositories, whereas FedAugment effectively minimizes the loss in prediction quality compared to the centralized setting.

5.4 Projection Efficiency

In a final series of experiments, we study the efficiency of FedAugment and our baselines. We distinguish between training and inference efficiency, and analyze time and space requirements. Table 5 summarizes training and inference time as well as GPU memory consumption for projection models trained on 12 embedding views and evaluated with an inference batch size of 32.

Training. We trained each method for up to 200 epochs or 24 hours. For FedAugment and vec2vec, we used a batch size of 4,096, which did not fully utilize the available 80 GB of GPU memory during training. LA2M does not rely on epoch-based training of a neural network architecture. When configured with PCA, LA2M exhibits the shortest training time, completing in under one hour on our 5% random training subset. Without PCA, however, the runtime of LA2M increases substantially due to the cost of clustering and Procrustes alignment at higher dimensionality, resulting in a training time of 4.4 hours, which already exceeds that of FedAugment. Since training is performed offline, we consider the training times of FedAugment and both LA2M variants to be practically feasible. In contrast, vec2vec reaches our 24-hour training limit and completes only 137 epochs within that budget. Its architecture employs three additional discriminators and optimizers per view, making it inherently inefficient for larger $\mathcal{P}$, which is typical in our decentralized setting. Thus, vec2vec's runtime stems not only from the number of trainable parameters but also from a large number of independent optimizers (cf. its comparably good inference performance). Prior work reports vec2vec training costs of up to seven GPU days with one million training instances per view [45], which is significantly more expensive than all other methods considered

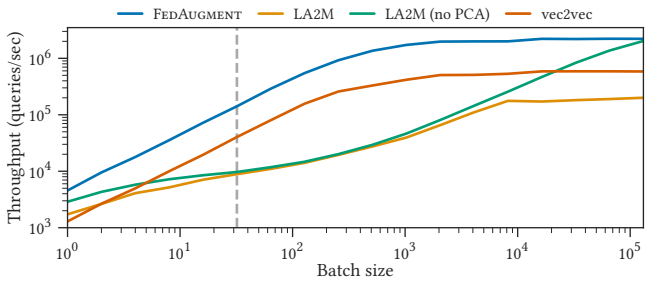


Figure 8: Projection inference throughput per batch size. The dashed vertical line indicates the batch size shown in Table 5.

here. Since vec2vec exceeds our training time budget, we did not perform extensive seed tuning. Independent of the chosen seed, our results indicate that vec2vec is not well-suited for our setting due to its instability, high training cost, and increased data requirements.

Inference. Projection model inference directly impacts the end-to-end latency in online table augmentation search. The batch size of 32 used in Table 5 reflects a realistic low-latency serving scenario. FedAugment achieves the lowest inference time at 0.2 ms while requiring the least GPU memory, outperforming both LA2M variants by more than an order of magnitude. This advantage stems from its computational structure: inference consists of a single matrix multiplication per layer over the entire batch. In contrast, LA2M must first assign each input to its nearest cluster and then apply a separate matrix multiplication for each activated cluster. LA2M without PCA attains slightly higher throughput than its PCA-based variant, but incurs two orders of magnitude higher memory consumption and approximately 6× longer training time. Besides projections, the HNSW retrieval time ranges from 0.4 to 0.8 ms on average, indicating that LA2M's projection overhead substantially affects end-to-end query latency. Figure 8 further analyzes inference scalability by reporting projection throughput across batch sizes from 10^0 to 10^5. FedAugment consistently compensates for its moderately higher training cost with up to two orders of magnitude higher inference throughput compared to LA2M. vec2vec and LA2M with PCA plateau at approximately 0.5 and one order of magnitude below FedAugment, respectively. Only LA2M without PCA matches FedAugment at a batch size of 2^{17}, when all clusters are activated, and the number of matrix multiplications no longer increases. Such batch sizes, however, are unrealistic for online serving. Moreover, the substantially higher GPU memory requirements translate into higher operational costs, as more powerful GPUs must be provisioned. Overall, FedAugment offers the most favorable efficiency trade-off, combining feasible offline training with clearly superior inference performance and low memory consumption.

6 RELATED WORK

Our work connects to prior research on embedding alignment, table representation learning, dataset search, and database integration.

Embedding Alignment. Aligning different vector spaces has been studied across multiple domains, most recently for vector database integration. Our evaluation shows that existing techniques, in particular LA2M [76] and vec2vec [45], are not well-suited to the heterogeneity and limited training data in decentralized settings. LA2M

learns an individual transformation from each source pipeline into a fixed reference space. It therefore cannot exploit the additional signal from larger sets of embedding spaces, which we leverage through contrastive learning. Compared to vec2vec, FedAugment only employs small projection heads that are more efficient and avoid the instability of GAN training. While vec2vec is fully unsupervised, our minimal assumptions on inference-accessible embedding pipelines make training data collection feasible. Embedding alignment in other domains, such as cross-lingual word translation [1, 47, 71] and text-to-image translation [66], typically focuses on aligning a small, fixed number of views and does not address the heterogeneity of decentralized data repositories.

Table Representation Learning. A large body of work develops table embedding models for downstream tasks such as question answering, text-to-SQL, and missing value imputation [23, 38, 39, 55, 78]. Recent surveys [3] and benchmarks [19] summarize advances in this area. FedAugment treats data providers' embedding pipelines as black boxes and enables interoperability across their heterogeneous outputs. Accordingly, methods that modify embedding models to enforce specific inductive biases, such as invariance to row or column permutations [37], are complementary.

Dataset Search Operators. Equi-join [8, 12, 30, 68, 81, 82], semantic join [18, 25, 26, 28, 37, 50], and union [8, 20, 33, 49, 57, 61] search have been extensively studied in centralized settings. Beyond table augmentation, prior work has proposed additional operators, such as query-by-example [17] and distribution-aware search [61], to enrich dataset discovery. All of these approaches assume either direct access to raw data or the availability of homogeneous data structures, such as aligned embeddings, knowledge graphs, or curated taxonomies. In contrast, FedAugment targets a setting without a central repository, where data providers independently deploy embedding pipelines and do not adhere to shared taxonomies.

Dataset Search Engines. Metadata-based search engines, including Google Dataset Search [63] and CKAN [65], primarily support keyword search and facet filtering. Fainder extends this paradigm with distribution-aware search [4] and a declarative query language [5]. Auctus [11] aggregates metadata from decentralized sources and downloads openly available datasets to enrich dataset profiles. Fully centralized systems such as Aurum [12] support advanced operators, including table augmentation, but rely on direct access to raw data, an assumption that we avoid.

Federated Database Integration. Conceptually, our setting relates to federated databases [69], where independent local databases must be mapped to a global schema. FedAugment's approach resembles a Global-as-View (GAV) approach [14, 52], as learned projection functions combine source embedding spaces into one global embedding space. Unlike classical GAV, where adding a new source requires rewriting the global view definition, we can train new projection functions incrementally. Moreover, we use only the global embedding space at query time to retrieve nearest neighbors and the corresponding dataset profiles without querying local repositories.

7 DISCUSSION

In the following, we discuss three key aspects of our framework and their implications for data providers and consumers.

Local vs. Global Top-k Retrieval. Instead of our approach, one could issue local top-k queries to each provider and merge the results. However, heterogeneous embedding pipelines make locally computed augmentation scores incomparable, so merging local results does not yield a well-defined global ranking. Without a globally consistent score, local top-k must retrieve $n \cdot k$ instead of k candidates to avoid missing results, thereby increasing transfer, processing, and data access costs. In contrast, we retrieve the global top-k from a unified embedding space via a single ANN query, improving efficiency and scalability because of sublinear index runtime.

Embedding Pipeline Accessibility. Despite our minimal assumptions about inference accessibility, data providers may not expose their embedding pipeline even for a fee due to technical or intellectual property constraints. In practice, provider-hosted pipeline access is not strictly required: specifying a machine-readable pipeline identifier is often sufficient, allowing the search engine to generate reference embeddings for third-party-hosted pipelines. For inaccessible pipelines, data providers can publish embeddings for a fixed set of reference columns to enable alignment.

Embedding Privacy. Adding embeddings to dataset profiles introduces privacy risks, as prior work shows that embeddings can leak information about their inputs [16, 45, 77]. In our setting, column embeddings reduce the risk of reconstructing complete records, since an attacker must also infer the correct row-level alignment. To further reduce exposure, providers can restrict access to dataset profiles to trusted search engines or perturb embeddings to balance privacy and utility. Overall, our framework does not eliminate the inherent privacy-discoverability trade-off.

8 CONCLUSIONS

We presented FedAugment, an end-to-end framework for table augmentation search over decentralized data repositories with heterogeneous embedding pipelines. Key components of FedAugment include the curation of training data, the learning of projection functions that align heterogeneous embedding spaces, and efficient vector similarity search over the global, aligned metadata. Our experimental evaluation demonstrates that FedAugment enables this new functionality while matching the effectiveness and efficiency of embedding-based augmentation search in centralized settings. We draw two main conclusions. First, embedding-based dataset search over decentralized data repositories is generally feasible, paving the way for practical deployment on existing decentralized repositories. Second, by retrieving only the global top-k datasets rather than a top-k result set from each data provider, FedAugment reduces the cost associated with dataset access and downstream processing. Interesting directions for future work include (1) improving projection models to mitigate deficiencies in embedding pipelines (e.g., sensitivity to row order), (2) enhancing augmentation search by combining embeddings with schema information and other metadata, and (3) designing specialized metadata representations tailored to join and union discovery.

ACKNOWLEDGMENTS

We gratefully acknowledge funding from the German Federal Ministry of Research, Technology and Space under the grants BIFOLD-25B and 01IS17052 (for the Software Campus project FDaaS).

REFERENCES

[1] M. Artetxe, G. Labaka, and E. Agirre. 2016. Learning Principled Bilingual Mappings of Word Embeddings While Preserving Monolingual Invariance. *EMNLP*. DOI: 10.18653/v1/D16-1250.

[2] Y. Babakhin, R. Osmulski, R. Ak, et al. 2025. Llama-Embed-Nemotron-8B: A Universal Text Embedding Model for Multilingual and Cross-Lingual Tasks. *arXiv*. DOI: 10.48550/arXiv.2511.07025.

[3] G. Badaro, M. Saeed, and P. Papotti. 2023. Transformers for Tabular Data Representation: A Survey of Models and Applications. *Trans. Assoc. Comput. Linguist.*, 11, 227–249. DOI: 10.1162/tacl_a_00544.

[4] L. Behme, S. Galhotra, K. Beedkar, et al. 2024. Fainder: A Fast and Accurate Index for Distribution-Aware Dataset Search. *Proc. VLDB Endow.*, 17, 11, 3269–3282. DOI: 10.14778/3681954.3681999.

[5] L. Behme, L. Geißler, P. Agrawal, et al. 2025. Finding What You're Looking For: A Distribution-Aware Dataset Search Engine in Action. *SIGMOD*. DOI: 10.1145/3722212.3725104.

[6] L. Behme, S. Thirumuruganathan, A. R. Mahdiraji, et al. 2023. The Art of Losing to Win: Using Lossy Image Compression to Improve Data Loading in Deep Learning Pipelines. *ICDE*. DOI: 10.1109/ICDE55515.2023.00077.

[7] O. Benjelloun, S. Chen, and N. Noy. 2020. Google Dataset Search by the Numbers. *ISWC*. DOI: 10.1007/978-3-030-62466-8_41.

[8] A. Bogatu, A. A. A. Fernandes, N. W. Paton, et al. 2020. Dataset Discovery in Data Lakes. *ICDE*. DOI: 10.1109/ICDE48307.2020.00067.

[9] A. Braud, G. Fromentoux, B. Radier, et al. 2021. The Road to European Digital Sovereignty with Gaia-X and IDSA. *IEEE Network*, 35, 2, 4–5. DOI: 10.1109/MNET.2021.9387709.

[10] B. V. Breugel and M. V. D. Schaar. 2024. Position: Why Tabular Foundation Models Should Be a Research Priority. *ICML*. Retrieved May 18, 2026 from https://proceedings.mlr.press/v235/van-breugel24a.html.

[11] S. Castelo, R. Rampin, A. Santos, et al. 2021. Auctus: A Dataset Search Engine for Data Discovery and Augmentation. *Proc. VLDB Endow.*, 14, 12, 2791–2794. DOI: 10.14778/3476311.3476346.

[12] R. Castro Fernandez, Z. Abedjan, F. Koko, et al. 2018. Aurum: A Data Discovery System. *ICDE*. DOI: 10.1109/ICDE.2018.00094.

[13] A. Chapman, E. Simperl, L. Koesten, et al. 2020. Dataset Search: A Survey. *VLDB J.*, 29, 1, 251–272. DOI: 10.1007/s00778-019-00564-x.

[14] S. S. Chawathe, H. Garcia-Molina, J. Hammer, et al. 1994. The TSIMMIS Project: Integration of Heterogeneous Information Sources. *IPSJ*. Retrieved May 18, 2026 from https://dbucsd.github.io/paperpdfs/1994_1.pdf.

[15] T. Chen, S. Kornblith, M. Norouzi, et al. 2020. A Simple Framework for Contrastive Learning of Visual Representations. *ICML*. Retrieved May 18, 2026 from https://proceedings.mlr.press/v119/chen20j.html.

[16] Y. Chen, Q. Xu, and J. Bjerva. 2025. ALGEN: Few-shot Inversion Attacks on Textual Embeddings via Cross-Model Alignment and Generation. *ACL*. DOI: 10.18653/v1/2025.acl-long.1185.

[17] M. P. Christensen, A. Leventidis, M. Lissandrini, et al. 2025. Fantastic Tables and Where to Find Them: Table Search in Semantic Data Lakes. *EDBT*. DOI: 10.48786/EDBT.2025.32.

[18] T. Cong, J. Gale, J. Frantz, et al. 2023. WarpGate: A Semantic Join Discovery System for Cloud Data Warehouses. *CIDR*. Retrieved May 18, 2026 from https://www.cidrdb.org/cidr2023/papers/p75-cong.pdf.

[19] T. Cong, M. Hulsebos, Z. Sun, et al. 2023. Observatory: Characterizing Embeddings of Relational Tables. *Proc. VLDB Endow.*, 17, 4, 849–862. DOI: 10.14778/3636218.3636237.

[20] T. Cong, F. Nargesian, and H. V. Jagadish. 2023. Pylon: Semantic Table Union Search in Data Lakes. *arXiv*. DOI: 10.48550/arXiv.2301.04901.

[21] Datarade. 2025. Find the Right Data Products, Effortlessly. Retrieved May 18, 2026 from https://datarade.ai.

[22] Dawex. 2025. Data Marketplaces, Data Hubs & Data Spaces. Retrieved May 18, 2026 from https://www.dawex.com/en/data-exchange-solution/#Discovery.

[23] X. Deng, H. Sun, A. Lees, et al. 2020. TURL: Table Understanding Through Representation Learning. *Proc. VLDB Endow.*, 14, 3, 307–319. DOI: 10.14778/3430915.3430921.

[24] Y. Deng, C. Chai, L. Cao, et al. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.*, 17, 8, 1925–1938. DOI: 10.14778/3659437.3659448.

[25] Y. Dong, K. Takeoka, C. Xiao, et al. 2021. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach. *ICDE*. DOI: 10.1109/ICDE51399.2021.00046.

[26] Y. Dong, C. Xiao, T. Nozawa, et al. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.*, 16, 10, 2458–2470. DOI: 10.14778/3603581.3603587.

[27] S. Elfwing, E. Uchibe, and K. Doya. 2018. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. *Neural Networks*, 107, 3–11. DOI: 10.1016/j.neunet.2017.12.012.

[28] M. Y. Eltabakh, M. Kunjir, A. K. Elmagarmid, et al. 2023. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. *Proc. VLDB Endow.*, 16, 11, 3377–3390. DOI: 10.14778/3611479.3611533.

[29] K. Enevoldsen, I. Chung, I. Kerboua, et al. 2024. MMTEB: Massive Multilingual Text Embedding Benchmark. *ICLR*. Retrieved May 18, 2026 from https://openreview.net/forum?id=zl3pfz4VCV.

[30] M. Esmailoghli, J.-A. Quiané-Ruiz, and Z. Abedjan. 2022. MATE: Multi-Attribute Table Extraction. *Proc. VLDB Endow.*, 15, 8, 1684–1696. DOI: 10.14778/3529337.3529353.

[31] M. Esmailoghli, C. Schnell, R. J. Miller, et al. 2025. BLEND: A Unified Data Discovery System. *ICDE*. DOI: 10.1109/ICDE65448.2025.00061.

[32] European Union. 2026. The official portal for European data. Retrieved May 18, 2026 from https://data.europa.eu/en.

[33] G. Fan, J. Wang, Y. Li, et al. 2023. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning. *Proc. VLDB Endow.*, 16, 7, 1726–1739. DOI: 10.14778/3587136.3587146.

[34] A. Fradkin. 2025. Demand for LLMs: Descriptive Evidence on Substitution, Market Expansion, and Multihoming. *arXiv*. DOI: 10.48550/arXiv.2504.15440.

[35] Y. Gong, G. Liu, Y. Xue, et al. 2023. A Survey on Dataset Quality in Machine Learning. *Inf. Software Technol.*, 162, 107268. DOI: 10.1016/j.infsof.2023.107268.

[36] C. Guo, B. Zhao, and Y. Bai. 2022. DeepCore: A Comprehensive Library for Coreset Selection in Deep Learning. *arXiv*. DOI: 10.48550/arXiv.2204.08499.

[37] Y. Guo, Y. Mao, Z. Hu, et al. 2025. Snoopy: Effective and Efficient Semantic Join Discovery via Proxy Columns. *IEEE Trans. Knowl. Data Eng.*, 37, 5, 2971–2985. DOI: 10.1109/TKDE.2025.3545176.

[38] J. Herzig, P. K. Nowak, T. Müller, et al. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. *ACL*. DOI: 10.18653/v1/2020.acl-main.398.

[39] N. Hollmann, S. Müller, K. Eggensperger, et al. 2023. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. *ICLR*. Retrieved May 18, 2026 from https://openreview.net/forum?id=cp5PvcI6w8_.

[40] S. Hong, H. Ryee, X. Jin, et al. 2025. How Organizations Choose Open-Source Generative AI Under Normative Uncertainty: The Moderating Role of Exploitative and Exploratory Behaviors. *J. Theor. Appl. Electron. Commer. Res.*, 20, 3, 250. DOI: 10.3390/jtaer20030250.

[41] H. Hu, X. Wang, Y. Zhang, et al. 2024. A Comprehensive Survey on Contrastive Learning. *Neurocomputing*, 610, 128645. DOI: 10.1016/j.neucom.2024.128645.

[42] M. Huh, B. Cheung, T. Wang, et al. 2024. Position: The Platonic Representation Hypothesis. *ICML*. Retrieved May 18, 2026 from https://proceedings.mlr.press/v235/huh24a.html.

[43] M. Hulsebos, Ç. Demiralp, and P. Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data*, 1, 1, 30:1–30:17. DOI: 10.1145/3588710.

[44] M. Hulsebos, W. Lin, S. Shankar, et al. 2024. It Took Longer than I Was Expecting: Why Is Dataset Search Still so Hard? *HILDA*. DOI: 10.1145/3665939.3665959.

[45] R. D. Jha, C. Zhang, V. Shmatikov, et al. 2025. Harnessing the Universal Geometry of Embeddings. *NeurIPS*. Retrieved May 18, 2026 from https://proceedings.neurips.cc/paper_files/paper/2025/file/4175dee33d6145cb8f0323703d138a53-Paper-Conference.pdf.

[46] I. T. Jolliffe. 2002. *Principal Component Analysis. Springer Series in Statistics*. Springer. DOI: 10.1007/b98835.

[47] A. Joulin, P. Bojanowski, T. Mikolov, et al. 2018. Loss in Translation: Learning Bilingual Word Mapping with a Retrieval Criterion. *EMNLP*. DOI: 10.18653/v1/D18-1330.

[48] Kaggle. 2024. Kaggle Datasets. Retrieved May 18, 2026 from https://www.kaggle.com/datasets.

[49] A. Khatiwada, G. Fan, R. Shraga, et al. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data*, 1, 1, 9:1–9:25. DOI: 10.1145/3588689.

[50] C. Koutras, J. Zhang, X. Qin, et al. 2025. OmniMatch: Joinability Discovery in Data Products. *Proc. VLDB Endow.*, 18, 11, 4588–4601. DOI: 10.14778/3749646.3749715.

[51] A. Kusupati, G. Bhatt, A. Rege, et al. 2022. Matryoshka Representation Learning. *NeurIPS*. Retrieved May 18, 2026 from https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf.

[52] A. Y. Levy, A. O. Mendelzon, Y. Sagiv, et al. 1995. Answering Queries Using Views. *SIGMOD*. DOI: 10.1145/212433.220198.

[53] P. Li, X. Rao, J. Blase, et al. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. *ICDE*. DOI: 10.1109/ICDE51399.2021.00009.

[54] Z. Li, X. Zhang, Y. Zhang, et al. 2023. Towards General Text Embeddings with Multi-stage Contrastive Learning. *arXiv*. DOI: 10.48550/arXiv.2308.03281.

[55] Q. Liu, B. Chen, J. Guo, et al. 2022. TAPEX: Table Pre-training via Learning a Neural SQL Executor. *ICLR*. Retrieved May 18, 2026 from https://openreview.net/forum?id=O50443AsCP.

[56] Y. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42, 4, 824–836. DOI: 10.1109/TPAMI.2018.2889473.

[57] M. Martorana, T. Kuhn, and J. van Ossenbruggen. 2025. Metadata-Driven Table Union Search: Leveraging Semantics for Restricted Access Data Integration. *arXiv*. DOI: 10.48550/arXiv.2502.20945.

[58] M. Maynou, S. Nadal, R. Panadero, et al. 2024. FREYJA: Efficient Join Discovery in Data Lakes. *arXiv*. DOI: 10.48550/arXiv.2412.06637.

[59] S. Mohammed, L. Budach, M. Feuerpfeil, et al. 2025. The Effects of Data Quality on Machine Learning Performance on Tabular Data. *Inf. Syst.*, 132, 102549. DOI: 10.1016/j.is.2025.102549.

[60] B. B. Moser, A. S. Shanbhag, S. Frolov, et al. 2025. A Coreset Selection of Coreset Selection Literature: Introduction and Recent Advances. *arXiv*. DOI: 10.48550/arXiv.2505.17799.

[61] F. Nargesian, E. Zhu, K. Q. Pu, et al. 2018. Table Union Search on Open Data. *Proc. VLDB Endow.*, 11, 7, 813–825. DOI: 10.14778/3192965.3192973.

[62] J. Ni, C. Qu, J. Lu, et al. 2022. Large Dual Encoders Are Generalizable Retrievers. *EMNLP*. DOI: 10.18653/V1/2022.EMNLP-MAIN.669.

[63] N. Noy, M. Burgess, and D. Brickley. 2019. Google Dataset Search: Building a Search Engine for Datasets in an Open Web Ecosystem. *WWW*. DOI: 10.1145/3308558.3313685.

[64] A. van den Oord, Y. Li, and O. Vinyals. 2019. Representation Learning with Contrastive Predictive Coding. *arXiv*. DOI: 10.48550/arXiv.1807.03748.

[65] Open Knowledge Foundation. 2026. CKAN - The Open Source Data Management System. Retrieved May 18, 2026 from http://ckan.org.

[66] A. Radford, J. W. Kim, C. Hallacy, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. *ICML*. Retrieved May 18, 2026 from https://proceedings.mlr.press/v139/radford21a.html.

[67] N. Reimers and I. Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. *EMNLP*. DOI: 10.18653/v1/D19-1410.

[68] A. Santos, A. Bessa, F. Chirigati, et al. 2021. Correlation Sketches for Approximate Join-Correlation Queries. *SIGMOD*. DOI: 10.1145/3448016.3458456.

[69] A. P. Sheth and J. A. Larson. 1990. Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases. *ACM Comput. Surv.*, 22, 3, 183–236. DOI: 10.1145/96602.96604.

[70] A. Shidani, D. Hjelm, J. Ramapuram, et al. 2024. Poly-View Contrastive Learning. *ICLR*. Retrieved May 18, 2026 from https://openreview.net/forum?id=iHcTLIor0m.

[71] S. L. Smith, D. H. P. Turban, S. Hamblin, et al. 2017. Offline Bilingual Word Vectors, Orthogonal Transformations and the Inverted Softmax. *ICLR*. Retrieved May 18, 2026 from https://openreview.net/forum?id=r1Aab85gg.

[72] P. Sun, D. Simcha, D. Dopson, et al. 2023. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. *NeurIPS*. Retrieved May 18, 2026 from https://proceedings.neurips.cc/paper_files/paper/2023/file/0973524e02a712af33325d0688ae6f49-Paper-Conference.pdf.

[73] Y. Tian, D. Krishnan, and P. Isola. 2020. Contrastive Multiview Coding. *ECCV*. DOI: 10.1007/978-3-030-58621-8_45.

[74] J. Traub, Z. Kaoudi, J.-A. Quiané-Ruiz, et al. 2021. Agora: Bringing Together Datasets, Algorithms, Models and More in a Unified Ecosystem. *SIGMOD Rec.*, 49, 4, 6–11. DOI: 10.1145/3456859.3456861.

[75] V. V. Vazirani. 2003. K-Center. In *Approximation Algorithms*. Springer, 47–53. DOI: 10.1007/978-3-662-04565-7_5.

[76] B. Yang, Y. Cao, and Y. Ren. 2025. Integrating Vector Databases across Embedding Models. *Proc. ACM Manag. Data*, 3, 6, 338:1–338:28. DOI: 10.1145/3769803.

[77] W. Yang, S. Wang, D. Wu, et al. 2025. Deep Learning Model Inversion Attacks and Defenses: A Comprehensive Survey. *Artif. Intell. Rev.*, 58, 8, 242. DOI: 10.1007/s10462-025-11248-0.

[78] P. Yin, G. Neubig, W.-t. Yih, et al. 2020. TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. *ACL*. DOI: 10.18653/v1/2020.acl-main.745.

[79] Y. Zhang, M. Li, D. Long, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv*. DOI: 10.48550/arXiv.2506.05176.

[80] X. Zhao, X. Hu, Z. Shan, et al. 2025. KaLM-Embedding-V2: Superior Training Techniques and Data Inspire a Versatile Embedding Model. *arXiv*. DOI: 10.48550/arXiv.2506.20923.

[81] E. Zhu, D. Deng, F. Nargesian, et al. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. *SIGMOD*. DOI: 10.1145/3299869.3300065.

[82] E. Zhu, F. Nargesian, K. Q. Pu, et al. 2016. LSH Ensemble: Internet-Scale Domain Search. *Proc. VLDB Endow.*, 9, 12, 1185–1196. DOI: 10.14778/2994509.2994534.